NOVA SCHOOL OF
SCIENCE & TECHNOLOGY

STR

TP3 – LAB#2

Petri Nets

2022 – 2023

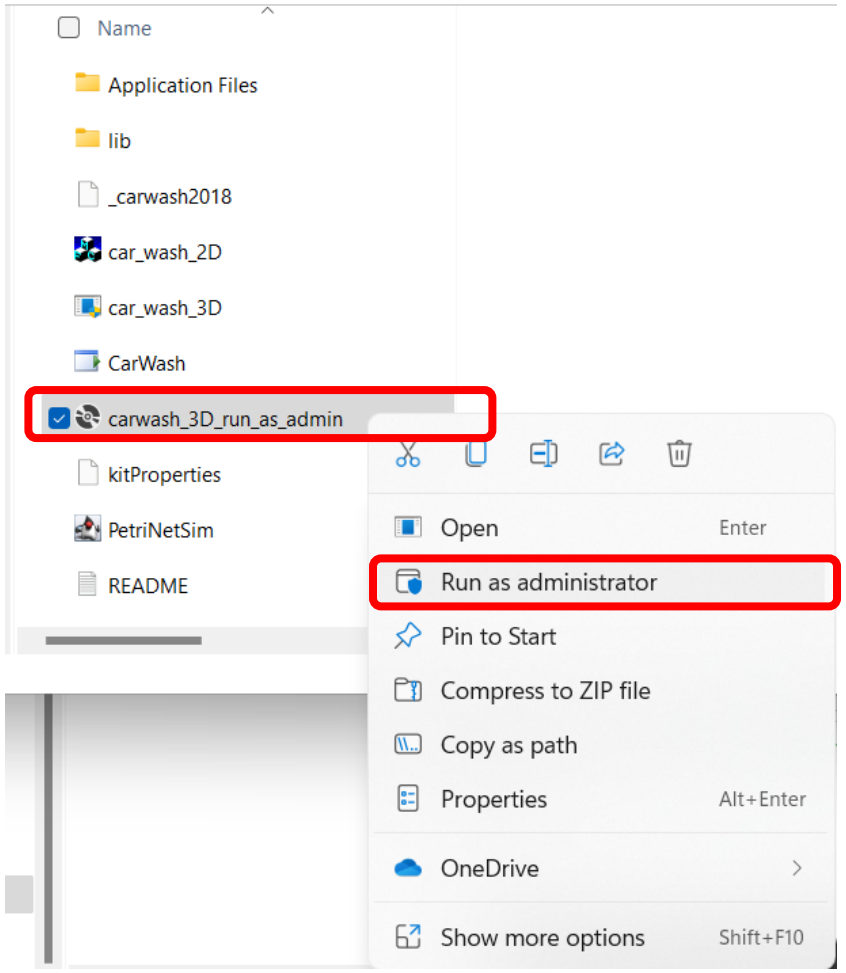
TOPICS

- Installing car wash simulator
- Project DLL interaction with HPSim
- User Interface

Car Wash Simulator

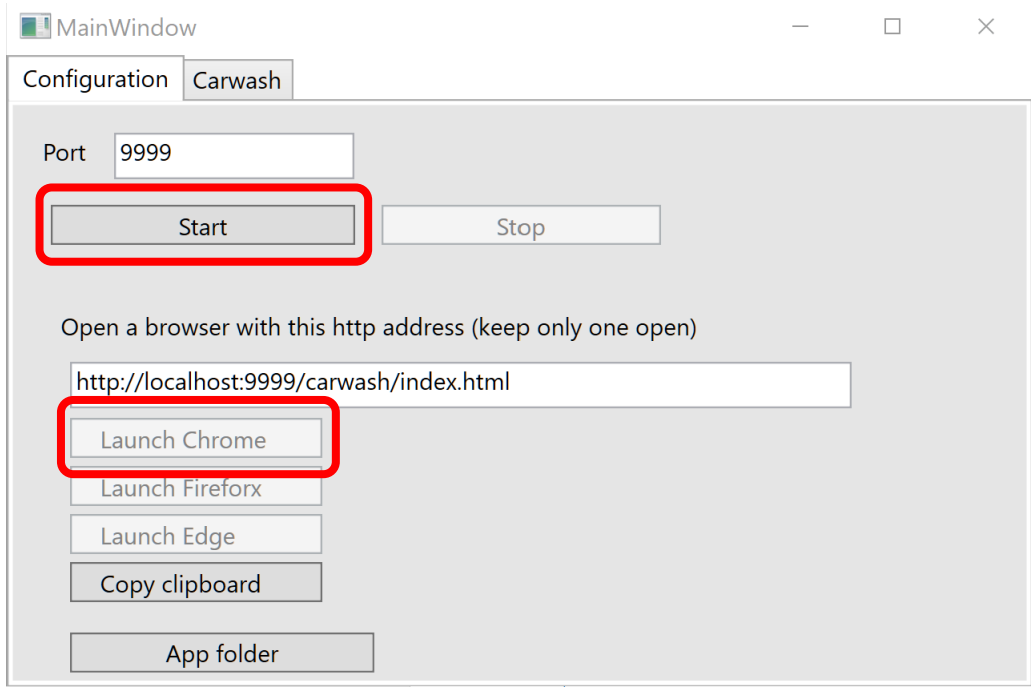
CAR WASH SIMULATOR INSTALLATION

- Download **carwash_simulator.zip** from CLIP and extract into **c:\str\labwork3\carwash**.
- Run as administrator the file highlighted in the picture.
- If asked to allow to make changes to your device click “yes”.
- If asked to assure you want to install the application, click “Install”.
- At the end, the simulator can be removed in control panel.



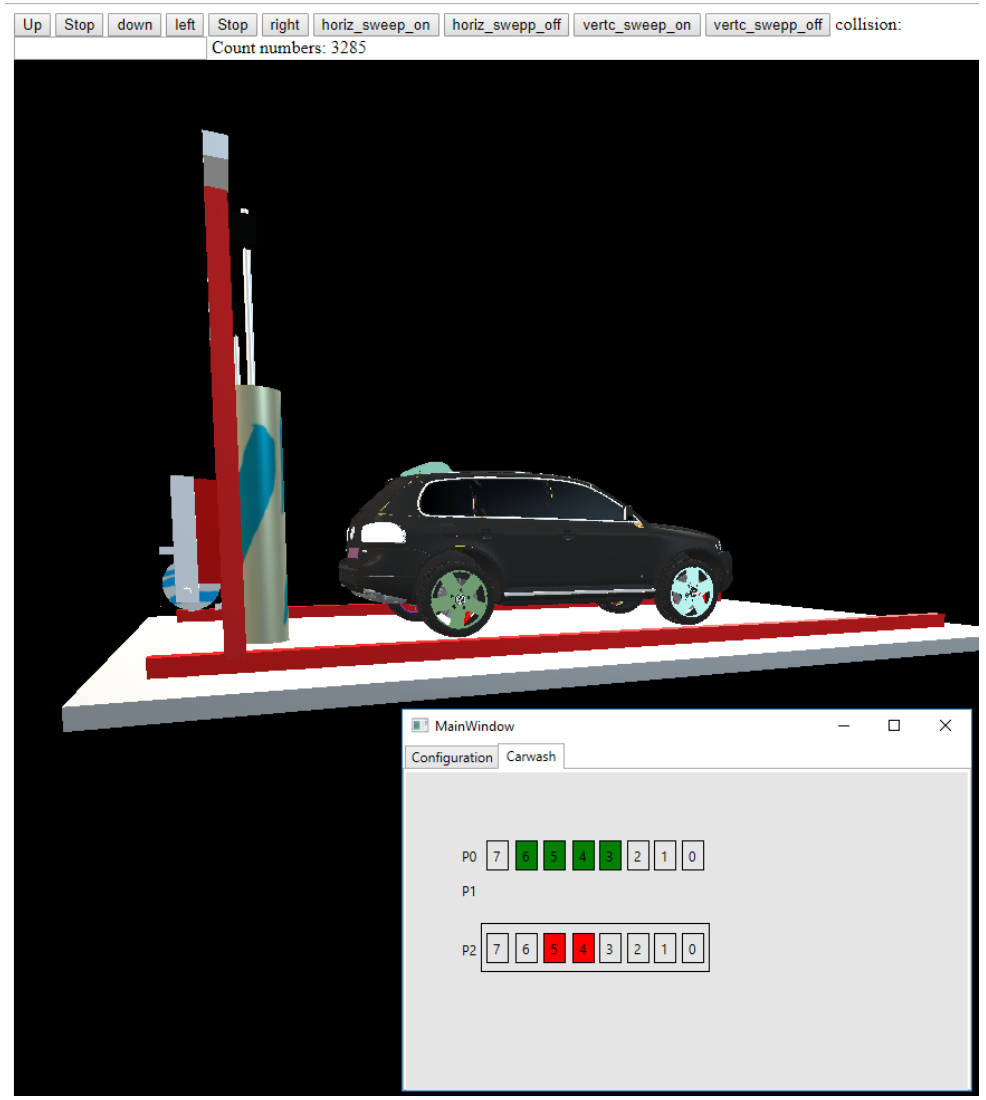
RUNNING CAR WASH SIMULATOR

- Subsequently, always run the simulator as administrator in the mentioned executable or in start menu.
- Click in start and in e.g., launch Chrome.
- If port 9999 is occupied choose other.



RUNNING THE CAR WASH SIMULATOR

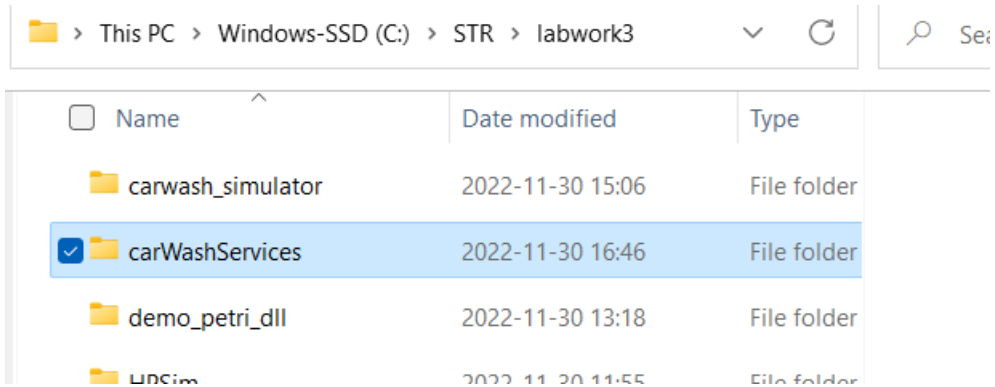
- Start interacting with the simulator.
- Change perspective and zoom in/out.
- Click on the bit buttons (the red ones) to get insights on its behavior.
- Like in the previous simulator, it follows stateless server approach, it is not necessary to re-start the simulator case of bugs in your project.

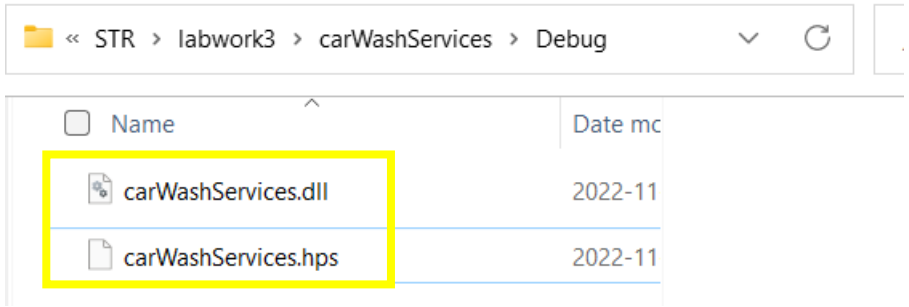


Project DLL Interaction with HPSim

CAR WASH SERVICES PROJECT

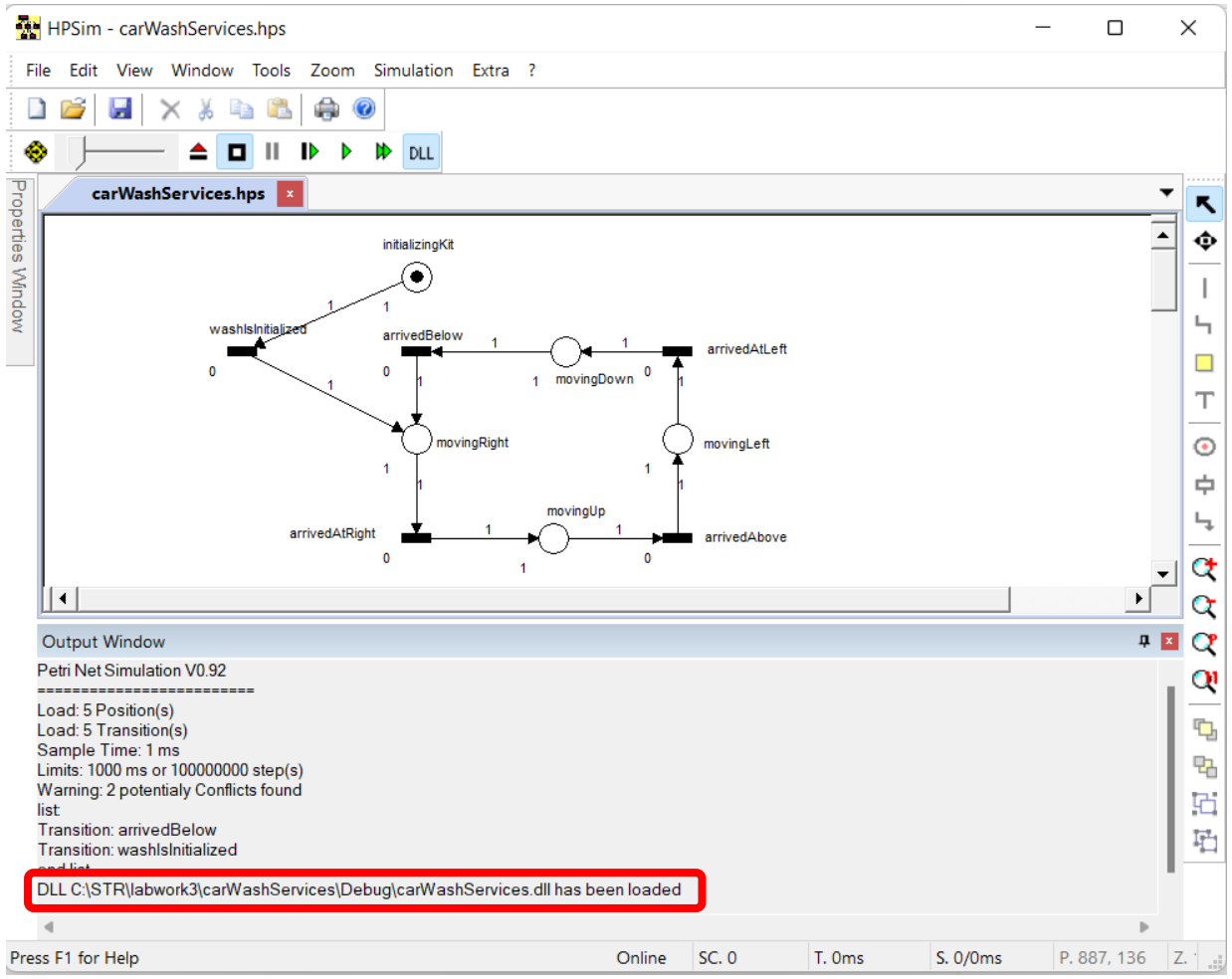
- The lab work is more focused on Petri Net modeling than programming.
- As such, a project with kit interaction functions is already provided.
- Download the package **carWashServices.zip** from CLIP and extract in c:\str\labwork3
- A Petri Net model named **carWashServices.hps** is also included inside the **debug** folder.
- As mentioned in previous lesson, they must have the same name, otherwise, DLL will not work for the PN model.





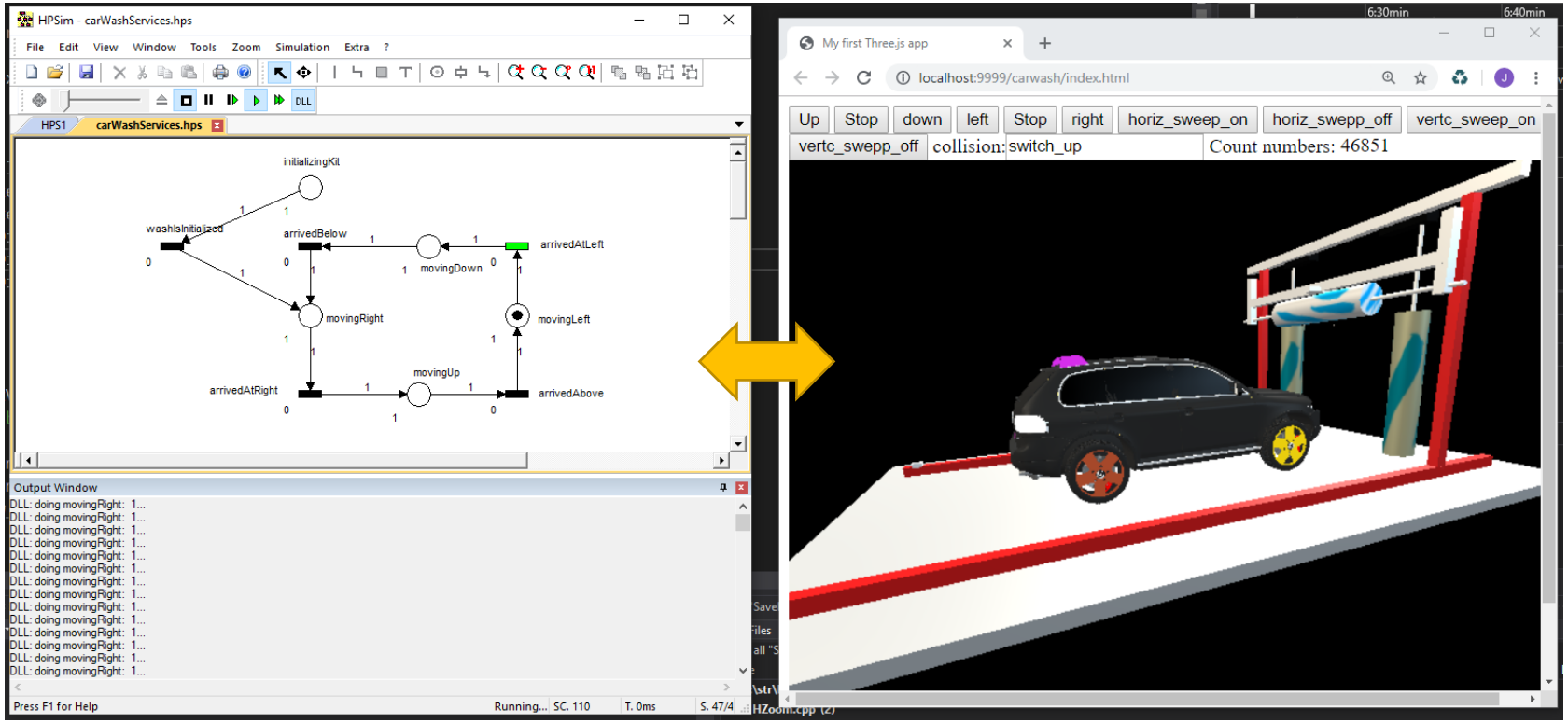
OPEN carWashServices.hps WITH HPSim

- As explained in previous lesson, it should enter in simulation mode (square button)
- Then, Press the DLL button.
- The DLL should have been loaded.
- (Pressing again on the DLL button, closes the DLL)



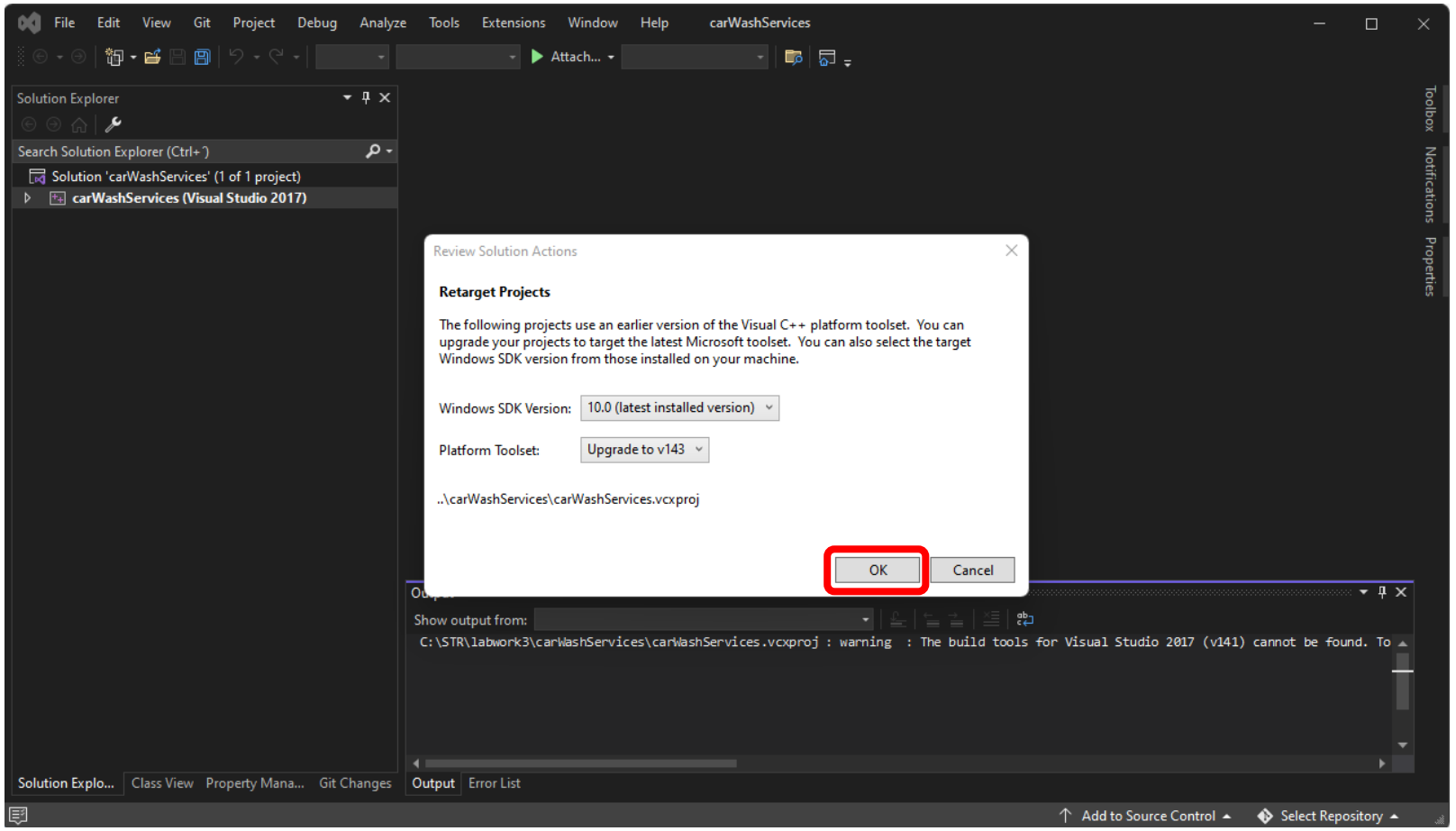
STARTING EXECUTION

- Pressing the play button (green triangular button), you should see the vertical brush moving right, up, left and down.
- The PN is continuously interacting with the functions in the DLL, imposing the behavior seen in the washing station.



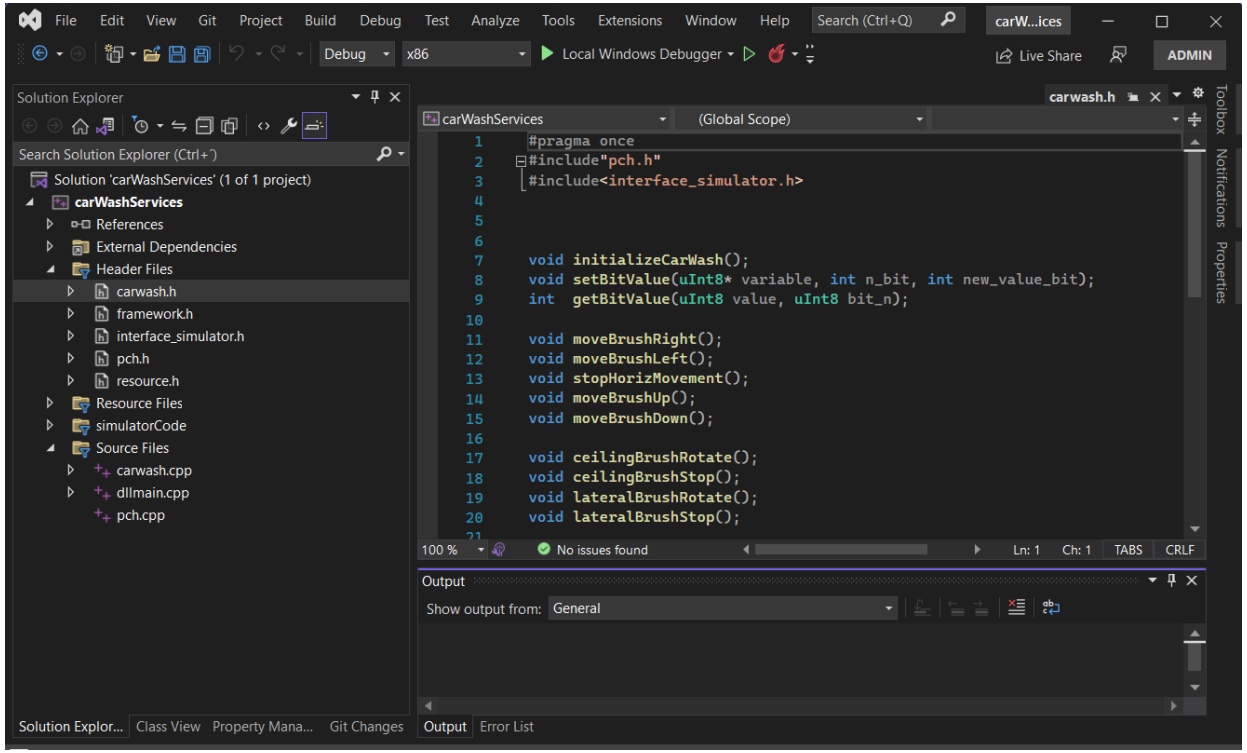
PLAY WITH YOUR PN

- Open the carWashServices project in Visual Studio.
- In case you are using more recent versions VS, you will be asked to retarged the project. Click OK!

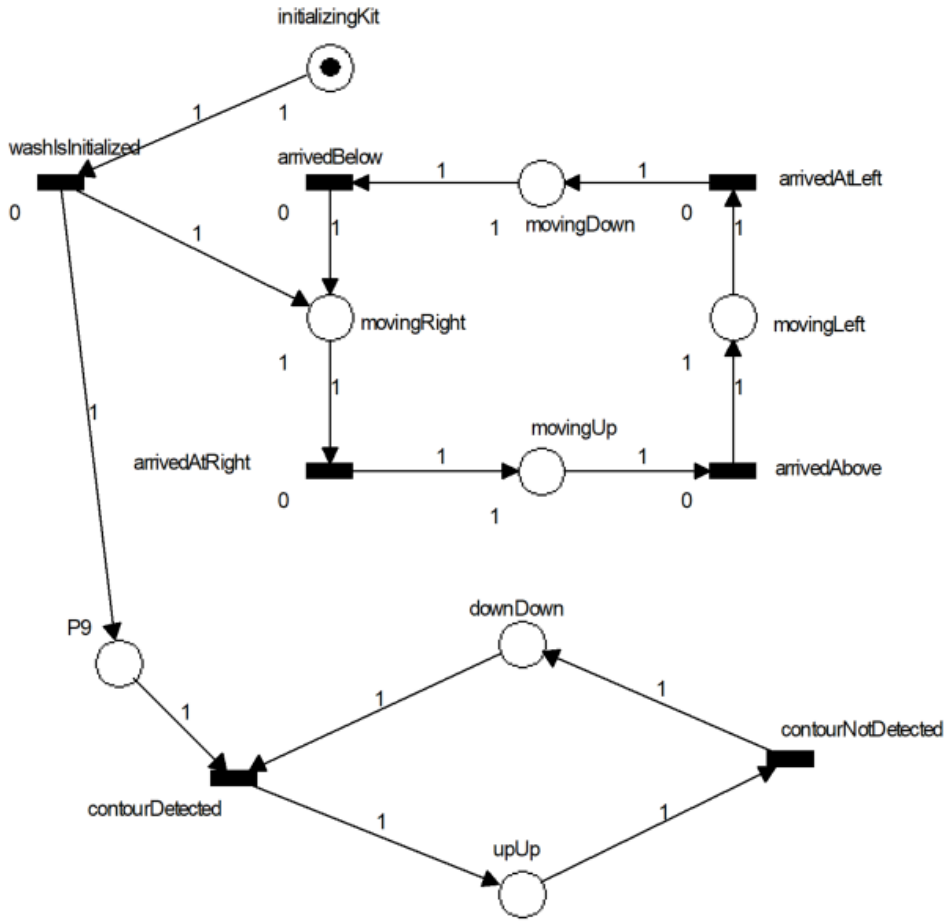


PLAY WITH YOUR PN

- Explore the carwash functionalities in carwash.h / carwash.cpp
- Using the available functions, add more transitions and places to the PN, so that you can manipulate the brushes and feel the contours of the car.
- Be careful, copy-paste each new transitions or places' names from carWashServices.hps properties to VS or vice-versa. That is, write the names in one side and copy-paste to the other (avoiding mistakes).
- Each time the C++ code is changed, compile the project (press DLL button to detach DLL, to avoid write permission errors).



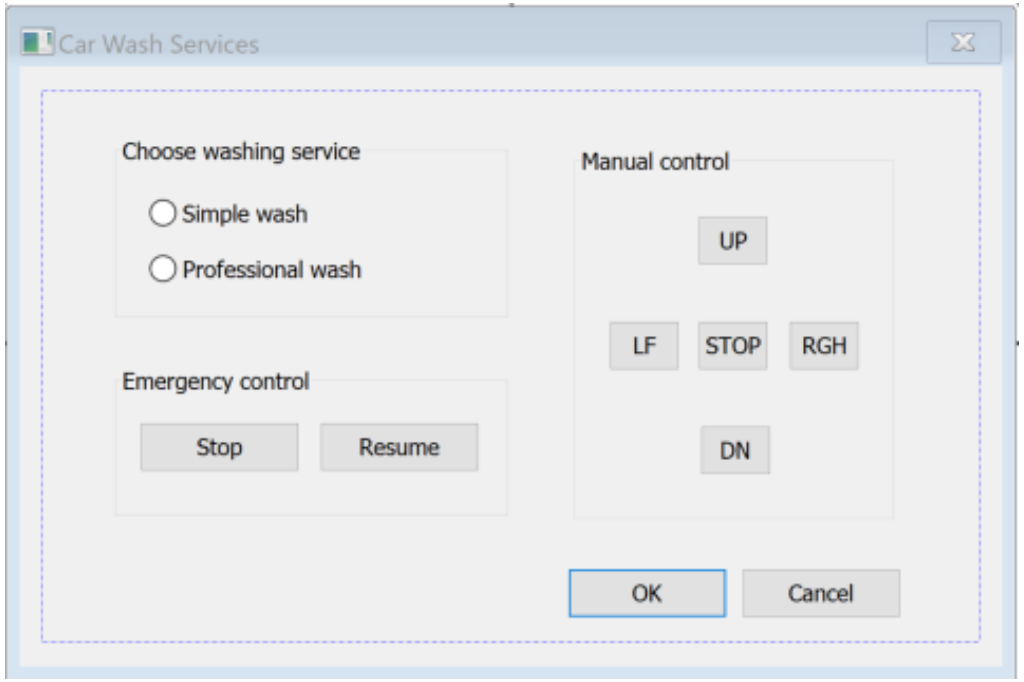
A SUGGESTION… TO START…



User Interface

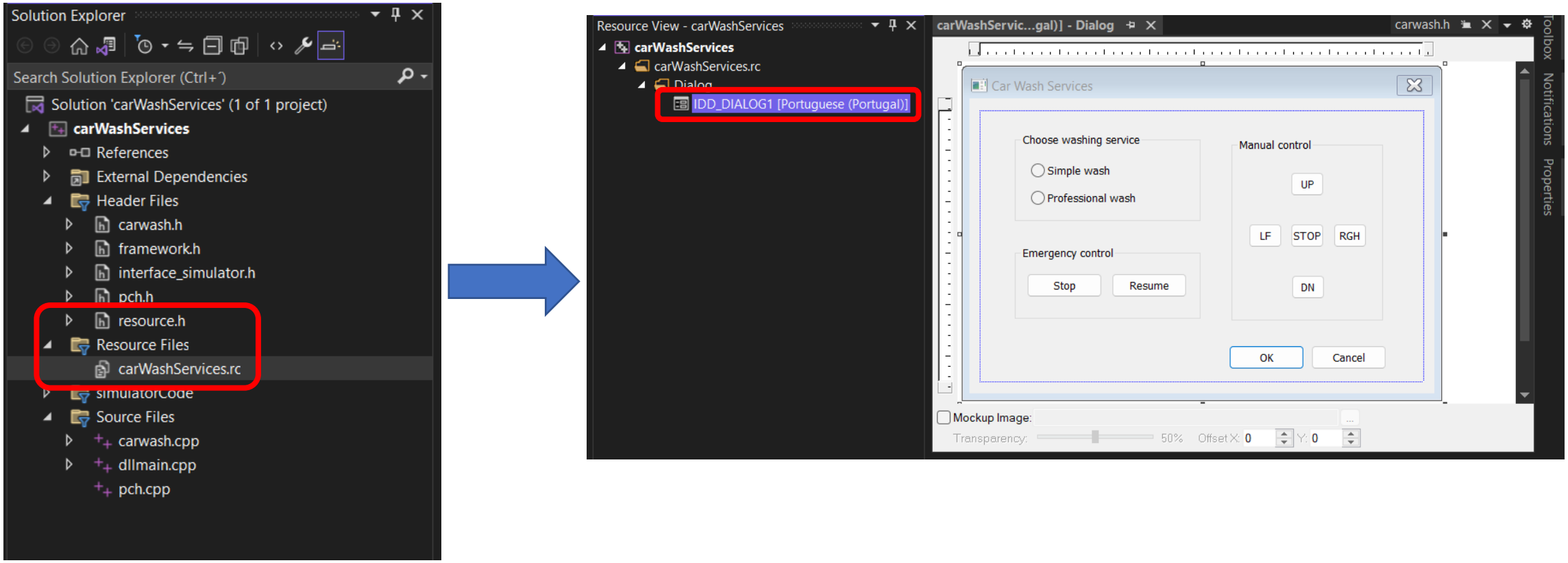
CAR WASH SERVICES UI

- The version of the **carWashServices** VS project, available in CLIP, provides a user interface (please feel free to change it / enhance it).



VS CONTEXT

- It can be found under the Resource Files folder -> carWashServices.rc



- The components IDs are defined in the **resource.h** file (explore it!).

UI – DLL INTEGRATION

- Copy-paste the lines below and add to **dllmain.cpp** at the position illustrated in the figure.

```
#include"resource.h"

HINSTANCE g_hInst = NULL;

extern "C" bool dialogLaunch();
```

```
4
5   #include<hpSimHandlers.h>
6   #include<carwash.h>
7
8   #include"resource.h"
9
10  HINSTANCE g_hInst = NULL;
11
12  extern "C" bool dialogLaunch();
13
```

Don’t worry (for now) with the error, in a couple of slides below this will be handled.

UI – DLL INTEGRATION (2)

- Call **dialogLaunch()** function inside the **initializingKit** handler.

```
67    //extern "C" __declspec(dllexport) void __cdecl
68    PLACE_HANDLER  initializingKit(long tokens)
69    {
70
71        initializationFinished = false;
72        initializeCarWash();
73        initializationFinished = true;
74
75        dialogLaunch();
76
77        // if returning false,
78        // visits HANDLER at every step
79        return true;
80    }
```

UI – DLL INTEGRATION (3)

- Creating the thread for launching the user interface.
- Copy-paste the code below and place it just above the DllMain function, as illustrated in the picture

```
HANDLE myHandle; // these declarations should be placed at the begining
HWND hwndGoto;

DWORD WINAPI myThread(LPVOID lpParameter){
    DialogBox(g_hInst, MAKEINTRESOURCE(IDD_DIALOG1), NULL, (DLGPROC)DialogProc);
    return 0;
}

extern "C" bool dialogLaunch(){

    DWORD myThreadID;

    myHandle = CreateThread(0, 0, myThread, NULL , 0, &myThreadID);
    return true;
}
```

```
164
165  HANDLE myHandle; // these declarations should be placed at the begining
166  HWND hwndGoto;
167
168 DWORD WINAPI myThread(LPVOID lpParameter) {
169      DialogBox(g_hInst, MAKEINTRESOURCE(IDD_DIALOG1), NULL, (DLGPROC)DialogProc);
170      return 0;
171  }
172
173 extern "C" bool dialogLaunch() {
174
175      DWORD myThreadID;
176
177      myHandle = CreateThread(0, 0, myThread, NULL, 0, &myThreadID);
178      return true;
179  }
180
```

```
extern "C" INT_PTR CALLBACK DialogProc(HWND hDlg, UINT uMsg, WPARAM wParam, LPARAM lParam) {
   int x = 0;
   switch (uMsg) {
   case WM_COMMAND:
       switch (LOWORD(wParam)) {
       case IDOK:
          if (IsDlgButtonChecked(hDlg, IDC_SIMPLE))
              selectedWash = IDC_SIMPLE;
          if (IsDlgButtonChecked(hDlg, IDC_PRO))
              selectedWash = IDC_PRO;
              EndDialog(hDlg, IDOK);
          break;
       case IDCANCEL:
          selectedWash = 0;
          manualCommand = 0;
          EndDialog(hDlg, IDCANCEL);
          break;
      case IDC_BUTTON_DOWN:
      case IDC_BUTTON_LEFT:
      case IDC_BUTTON_UP:
      case IDC_BUTTON_RIGHT:
      case IDC_BUTTON_STOP:
         manualCommand = LOWORD(wParam);
         break;
      case IDC_SIMPLE:
      case IDC_PRO:
         selectedWash = LOWORD(wParam);
       }
       break;
   case WM_CLOSE:
      ::EndDialog(hDlg, IDOK);
      return TRUE;
   case WM_DESTROY:
      DestroyWindow(hDlg);
      PostQuitMessage(0);
      break;
   default:
      return FALSE;
   }
   return TRUE;
}
```

- Code for the callback function.
- Copy-paste the code on the left and place it just above the myThread function:

```
164
165 extern "C" INT_PTR CALLBACK DialogProc(HWND hDlg, UINT uMsg, WPARAM wParam, LPARAM lParam) {
166     int x = 0;
167     switch (uMsg) {
168     case WM_COMMAND:
169         switch (LOWORD(wParam)) {
170         case IDOK:
171             if (IsDlgButtonChecked(hDlg, IDC_SIMPLE))
172                 selectedWash = IDC_SIMPLE;
173             if (IsDlgButtonChecked(hDlg, IDC_PRO))
174                 selectedWash = IDC_PRO;
175             EndDialog(hDlg, IDOK);
176             break;
177         case IDCANCEL:
178             selectedWash = 0;
179             manualCommand = 0;
180             EndDialog(hDlg, IDCANCEL);
181             break;
182         case IDC_BUTTON_DOWN:
183         case IDC_BUTTON_LEFT:
184         case IDC_BUTTON_UP:
185         case IDC_BUTTON_RIGHT:
186         case IDC_BUTTON_STOP:
```

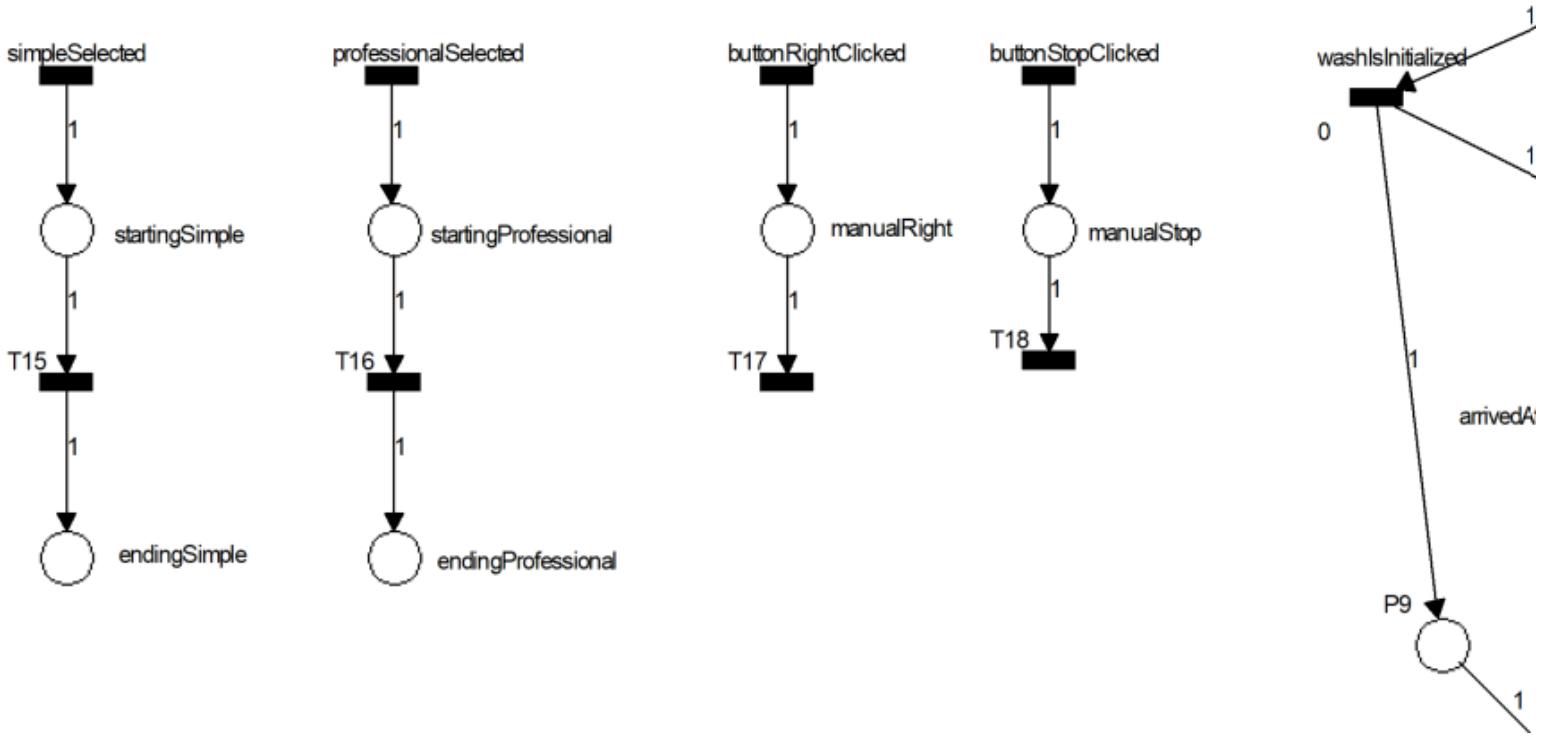
UI – DLL INTEGRATION (5)

- Finally, change the DllMain function, as illustrated in figure below.

```
225  BOOL APIENTRY DllMain(HMODULE hModule,
226      DWORD  ul_reason_for_call,
227      LPVOID lpReserved
228  )
229  {
230      switch (ul_reason_for_call)
231      {
232      case DLL_PROCESS_ATTACH:
233          g_hInst = hModule;
234      case DLL_THREAD_ATTACH:
235      case DLL_THREAD_DETACH:
236      case DLL_PROCESS_DETACH:
237          break;
238      }
239      return TRUE;
240  }
```

EXTEND THE PETRI NET

- The examples below illustrate the approach on how to receive commands from the UI.
- Develop the network (just for the sake of testing…).



HANDLERS FOR THE NEW TRANSITIONS

- Copy-paste the code at right for handling the new transitions.

```
TRANSITION_HANDLER  simpleSelected() {
   return selectedWash == IDC_SIMPLE;
}

TRANSITION_HANDLER  professionalSelected() {
   return selectedWash == IDC_PRO;
}

TRANSITION_HANDLER  buttonRightClicked() {
   return manualCommand == IDC_BUTTON_RIGHT;
}

TRANSITION_HANDLER  buttonStopClicked() {
   return manualCommand == IDC_BUTTON_STOP;
}
```

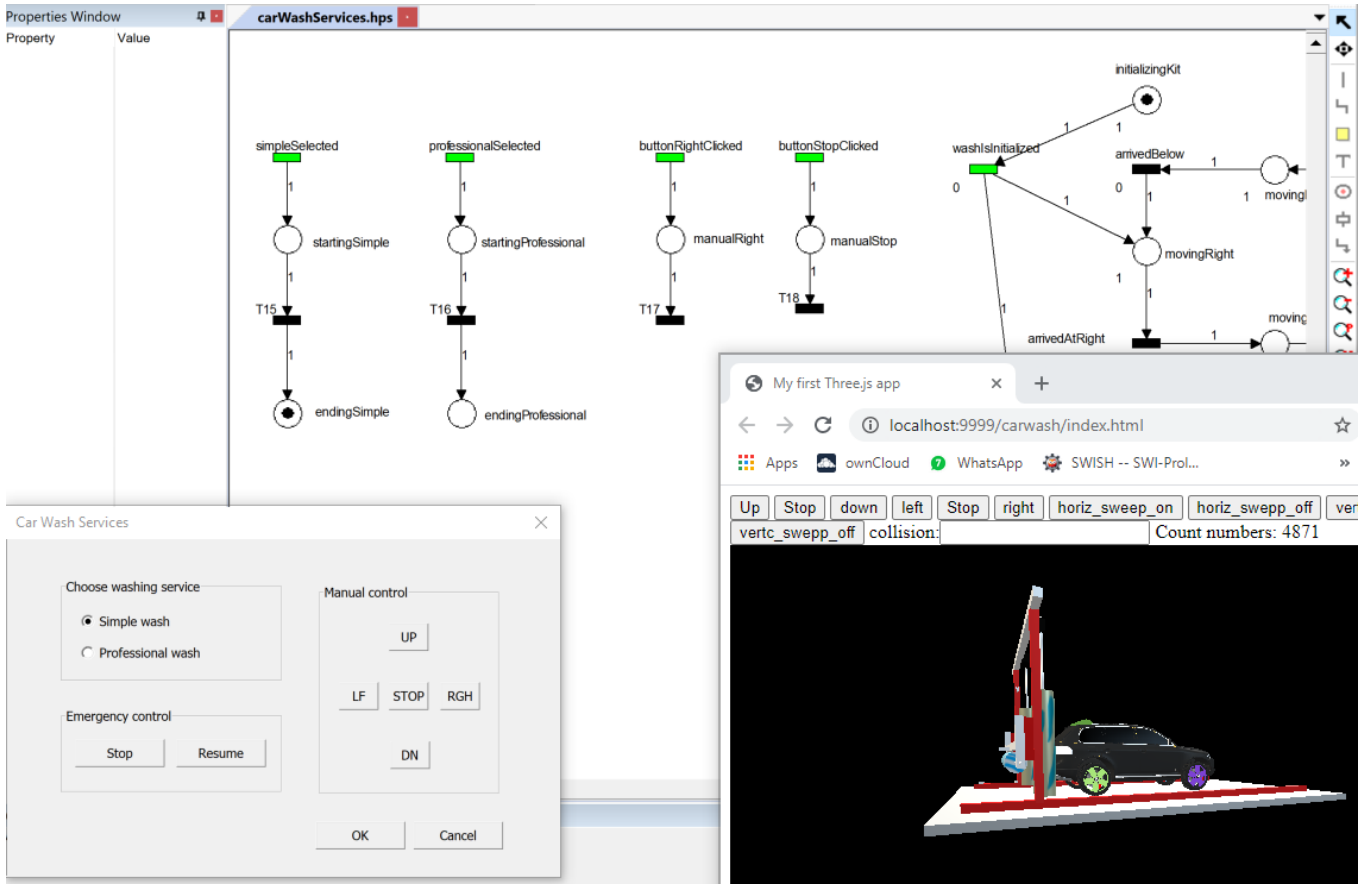
HANDLERS FOR THE NEW PLACES

- Copy-paste the code at right for handling the new places.

```
PLACE_HANDLER manualRight(long tokens) {
    moveBrushRight();
    manualCommand = 0;
    return true;
}

PLACE_HANDLER manualStop(long tokens) {
    stopHorizMovement();
    stopVerticalMovement();
    manualCommand = 0;
    return true;
}

PLACE_HANDLER endingSimple(long tokens) {
    selectedWash = 0;
    return true;
}

PLACE_HANDLER endingProfessional(long tokens) {
    selectedWash = 0;
    return true;
}
```

BUILD AND TEST

- Build the DLL.
- Start the execution.
- Now click on the services selection or manual control buttons.
- You should see a token appearing for the selected option.
- The manual right button and stop functionality is already provided; try it.



FINAL REMARKS

- Finish the PN diagram for managing the car wash station.
- Integrate with the DLL and UI functionality.

- This comprises everything to finish it all.
- Enjoy!!